

ASSIGNMENT-6.1

Hall Ticket: 2303A510i2

Name: Kasuri Rohith

Batch:23

Task Description

1 (Loops – Automorphic Numbers in a Range) • Task: Prompt AI to generate a function that displays all Automorphic numbers between 1 and 1000 using a for loop.

- Instructions:

Get AI-generated code to list Automorphic numbers using a for loop. o Analyse the correctness and anciency of the generated logic. o Ask AI to regenerate using a while loop and compare both implementations.

Expected Output #1: Correct implementation that lists Automorphic numbers using both loop types, with explanation

```
1 #generate all automorphic numbers within the range of 1 to 1000 using for loop
2 import time as t
3 def is_automorphic(num):
4     square = num * num
5     num_str = str(num)
6     square_str = str(square)
7     return square_str.endswith(num_str)
8
9 start_time = t.time()
10 for i in range(1, 1001):
11     num_str = str(i)
12     square_str = str(i * i)
13     if is_automorphic(i):
14         print(f"{i} is an automorphic number.")
15 end_time = t.time()
16 print(f"Execution Time: {end_time - start_time} seconds")
17
18 #generate all automorphic numbers within the range of 1 to 1000 using while loop
19 import time as t
20 def is_automorphic(num):
21     square = num * num
22     num_str = str(num)
23     square_str = str(square)
24     return square_str.endswith(num_str)
25
26 start_time = t.time()
27 i = 1
28 while i <= 1000:
29     if is_automorphic(i):
30         print(f"{i} is an automorphic number.")
31     i += 1
32 end_time = t.time()
33 print(f"Execution Time: {end_time - start_time} seconds")
34
```

OUTPUT:

```
PS C:\Users\rohit\OneDrive\Desktop\ai> & C:/Users/rohit/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/rohit/OneDrive/Desktop/ai/new.py
1 is an automorphic number.
5 is an automorphic number.
6 is an automorphic number.
25 is an automorphic number.
76 is an automorphic number.
376 is an automorphic number.
625 is an automorphic number.
Execution Time: 0.02458786964416504 seconds
1 is an automorphic number.
5 is an automorphic number.
6 is an automorphic number.
25 is an automorphic number.
76 is an automorphic number.
376 is an automorphic number.
625 is an automorphic number.
Execution Time: 0.0014529228210449219 seconds
```

2.Task Description

(Conditional Statements – Online Shopping Feedback Classification)

- Task: Ask AI to write nested if-elif-else conditions to classify online shopping feedback as Positive, Neutral, or Negative based on a numerical rating (1–5).
- Instructions:
 - o Generate initial code using nested if-elif-else.
 - o Analyze correctness and readability.
 - o Ask AI to rewrite using dictionary-based or match-case structure.

Expected Output #2:

- Feedback classification function with explanation and an alternative approach.

```
# Generate feedback for shopping ratings using conditional statements and dictionary mapping
def shopping_feedback(rating):
    if rating > 3:
        return "positive"
    elif rating == 3:
        return "neutral"
    elif rating >= 1:
        return "negative"
    else:
        return "Invalid rating! Please provide a rating between 1 and 5."

user_rating = 2
feedback = shopping_feedback(user_rating)
print(f"User rating: {user_rating} => Feedback: {feedback}")

def shopping_feedback_dict(rating):
    feedback_map = {
        5: "positive",
        4: "positive",
        3: "neutral",
        2: "negative",
        1: "negative"
    }
    return feedback_map.get(rating, "Invalid rating! Please provide a rating between 1 and 5.")

user_rating = 4
feedback = shopping_feedback_dict(user_rating)
print(f"User rating: {user_rating} => Feedback: {feedback}")
```

Output:

```
PS C:\Users\rohit\OneDrive\Desktop\ai> & C:/Users/rohit/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/rohit/OneDrive/Desktop/ai/new.py
User rating: 2 => Feedback: negative
User rating: 4 => Feedback: positive
PS C:\Users\rohit\OneDrive\Desktop\ai>
```

Task 3: Statistical_operations

Define a function named statistical_operations(tuple Num) that performs the following statistical operations on a tuple of numbers:

- Minimum, Maximum
- Mean, Median, Mode
- Variance, Standard Deviation

While writing the function, observe the code suggestions provided by GitHub Copilot. Make decisions to accept, reject, or modify the suggestions based on their relevance and correctness

```

64 # Write a Python program to perform statistical operations on a tuple of numbers without importing modules
65 data = (10, 20, 30, 40, 50, 60, 70, 80, 90, 100)
66
67 max_value = max(data)
68 min_value = min(data)
69 mean = sum(data) / len(data)
70
71 sorted_data = sorted(data)
72 n = len(sorted_data)
73
74 if n % 2 == 0:
75     median = (sorted_data[n//2 - 1] + sorted_data[n//2]) / 2
76 else:
77     median = sorted_data[n//2]
78
79 freq = {}
80 for num in data:
81     freq[num] = freq.get(num, 0) + 1
82
83 mode = max(freq, key=freq.get)
84
85 variance = sum((x - mean) ** 2 for x in data) / (n - 1)
86 stdev = variance ** 0.5
87
88 print(f>Data: {data}<)
89 print(f>Maximum Value: {max_value}<)
90 print(f>Minimum Value: {min_value}<)
91 print(f>Mean: {mean}<)
92 print(f>Median: {median}<)
93 print(f>Mode: {mode}<)
94 print(f>Standard Deviation: {stdev}<)
95 print(f>Variance: {variance}<)
96

```

OUTPUT:

```

PS C:\Users\rohit\OneDrive\Desktop\ai> & C:/Users/rohit/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/rohit/OneDrive/Desktop/ai/new.py
Median: 55.0
Maximum Value: 100
Minimum Value: 10
Mean: 55.0
Maximum Value: 100
Minimum Value: 10
Maximum Value: 100
Maximum Value: 100
Minimum Value: 10
Mean: 55.0
Median: 55.0
Mode: 10
Standard Deviation: 30.276503540974915
Variance: 916.6666666666666
PS C:\Users\rohit\OneDrive\Desktop\ai> & C:/Users/rohit/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/rohit/OneDrive/Desktop/ai/new.py

```

Task 4: Teacher Profile

- Prompt: Create a class Teacher with attributes teacher_id, name, subject, and experience.

Add a method to display teacher details.

- Expected Output: Class with initializer, method, and object creation.

```

97 # Write a Python program to create a class Teacher with attributes teacher_id, teacher_name,
98 # subject, and start_year
99
100 class Teacher:
101     def __init__(self, teacher_id, teacher_name, subject, start_year):
102         self.teacher_id = teacher_id
103         self.teacher_name = teacher_name
104         self.subject = subject
105         self.start_year = start_year
106
107     def calculate_experience(self, current_year):
108         return current_year - self.start_year
109
110 # Example usage
111 teacher1 = Teacher(1, "Alice Smith", "Mathematics", 2010)
112 current_year = 2024
113 experience = teacher1.calculate_experience(current_year)
114
115 print(f"Teacher ID: {teacher1.teacher_id}")
116 print(f"Teacher Name: {teacher1.teacher_name}")
117 print(f"Subject: {teacher1.subject}")
118 print(f"Years of Experience: {experience} years")
119

```

OUTPUT:

```

PS C:\Users\rohit\OneDrive\Desktop\ai> & C:/Users/rohit/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/rohit/OneDrive/Desktop/ai/new.py
Teacher ID: 1
Teacher Name: Alice Smith
Subject: Mathematics
Years of Experience: 14 years
PS C:\Users\rohit\OneDrive\Desktop\ai>

```

Task #5 – Zero-Shot Prompting with Conditional Validation

Use zero-shot prompting to instruct an AI tool to generate a function that validates an Indian mobile number.

Requirements

- The function must ensure the mobile number:
 - o Starts with 6, 7, 8, or 9
 - o Contains exactly 10 digits

Expected Output

- A valid Python function that performs all required validations without using any input-output examples in the prompt.

```
120 # Validate an Indian mobile number that starts with 6, 7, 8, or 9 and has exactly 10 digits
121
122 def validate_indian_mobile_number(mobile_number):
123     if len(mobile_number) == 10 and mobile_number.isdigit() and mobile_number[0] in ['6', '7', '8', '9']:
124         return True
125     else:
126         return False
127
128 mobile_number = input("Enter an Indian mobile number to validate: ")
129
130 if validate_indian_mobile_number(mobile_number):
131     print(f"{mobile_number} is a valid Indian mobile number.")
132 else:
133     print(f"{mobile_number} is not a valid Indian mobile number.")
134
135 # This program checks if the entered mobile number is valid according to Indian mobile number standards.
136 # It ensures the number is 10 digits long, consists only of digits, and starts with 6, 7, 8, or 9.
137
```

OUTPUT:

```
PS C:\Users\rohit\OneDrive\Desktop\ai> & C:/Users/rohit/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/rohit/OneDrive/Desktop/ai/new.py
Enter an Indian mobile number to validate: 9182506691
9182506691 is a valid Indian mobile number.
PS C:\Users\rohit\OneDrive\Desktop\ai>
```

Task Description #6 (Loops – Armstrong Numbers in a Range)

Task: Write a function using AI that finds all Armstrong numbers in a user- specified range (e.g., 1 to 1000).

Instructions:

- Use a for loop and digit power logic.
- Validate correctness by checking known Armstrong numbers (153, 370, etc.).
- Ask AI to regenerate an optimized version (using list comprehensions).

Expected Output #7:

- Python program listing Armstrong numbers in the range.
- Optimized version with explanation.


```

138 # Find Armstrong numbers within the range 1-1000 using a for loop and validate by checking known Armstrong numbers
139
140 def is_armstrong(num):
141     order = len(str(num))
142     sum_of_powers = sum(int(digit) ** order for digit in str(num))
143     return sum_of_powers == num
144
145 armstrong_numbers = []
146
147 for i in range(1, 1001):
148     if is_armstrong(i):
149         armstrong_numbers.append(i)
150
151 print("Armstrong numbers between 1 and 1000 are:", armstrong_numbers)
152 print("Validated Armstrong numbers:", [num for num in armstrong_numbers if is_armstrong(num)])
153
154

```

OUTPUT:

```

PS C:\Users\rohit\OneDrive\Desktop\ai> & C:/Users/rohit/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/rohit/OneDrive/Desktop/ai/new.py
Armstrong numbers between 1 and 1000 are: [1, 2, 3, 4, 5, 6, 7, 8, 9, 153, 370, 371, 407]
Validated Armstrong numbers: [1, 2, 3, 4, 5, 6, 7, 8, 9, 153, 370, 371, 407]
PS C:\Users\rohit\OneDrive\Desktop\ai>

```

Task Description #7 (Loops – Happy Numbers in a Range)

Task: Generate a function using AI that displays all Happy Numbers within a user-specified range (e.g., 1 to 500).

Instructions:

- Implement the logic using a loop: repeatedly replace a number with the sum of the squares of its digits until the result is either 1 (Happy Number) or enters a cycle (Not Happy).
- Validate correctness by checking known Happy Numbers (e.g., 1, 7, 10, 13, 19, 23, 28...).
- Ask AI to regenerate an optimized version (e.g., by using a set to detect cycles instead of infinite loops).

Expected Output #8:

- Python program that prints all Happy Numbers within a range.
- Optimized version using cycle detection with explanation.

```

156 def is_happy_number(num):
157     seen = set()
158     while num != 1 and num not in seen:
159         seen.add(num)
160         num = sum(int(digit) ** 2 for digit in str(num))
161     return num == 1
162
163 start_range = 1
164 end_range = 500
165 happy_numbers = []
166
167 for i in range(start_range, end_range + 1):
168     if is_happy_number(i):
169         happy_numbers.append(i)
170
171 print(f"Happy numbers between {start_range} and {end_range}: {happy_numbers}")
172

```

OUTPUT:

```

PS C:\Users\rohit\OneDrive\Desktop\ai> & C:/Users/rohit/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/rohit/OneDrive/Desktop/ai/new.py
Happy numbers between 1 and 500: [1, 7, 10, 13, 19, 23, 28, 31, 32, 44, 49, 68, 70, 79, 82, 86, 91, 94, 97, 100, 103, 109, 129, 130, 133, 139, 167, 176, 188, 190, 192, 193, 203, 208, 219, 226, 230, 236, 239, 262, 263, 280, 291, 293, 301, 302, 310, 313, 319, 320, 326, 329, 331, 338, 356, 362, 365, 367, 368, 376, 379, 383, 386, 391, 392, 397, 404, 409, 440, 446, 464, 469, 478, 487, 490, 496]
PS C:\Users\rohit\OneDrive\Desktop\ai>

```

Task Description #8 (Loops – Strong Numbers in a Range)

Task: Generate a function using AI that displays all Strong Numbers (sum of factorial of digits equals the number, e.g., $145 = 1! + 4! + 5!$) within a given range.

Instructions:

- Use loops to extract digits and calculate factorials.
- Validate with examples (1, 2, 145).
- Ask AI to regenerate an optimized version (precompute digit factorials).


```

172
173 # Display all strong numbers where the sum of factorial of digits equals the number
174 import time as t
175
176 def factorial(n):
177     if n == 0 or n == 1:
178         return 1
179     else:
180         return n * factorial(n - 1)
181
182 start_time = t.time()
183 i = 1
184
185 while i <= 1000:
186     sum_of_factorials = sum(factorial(int(digit)) for digit in str(i))
187     if sum_of_factorials == i:
188         print(f"{i} is a strong number.")
189     i += 1
190
191 end_time = t.time()
192 print(f"Execution Time: {end_time - start_time} seconds")
193
194 # This program defines a function to calculate the factorial of a number.
195

```

Output:

```

PS C:\Users\rohit\OneDrive\Desktop\ai> & C:/Users/rohit/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/rohit/OneDrive/Desktop/ai/new.py
1 is a strong number.
2 is a strong number.
145 is a strong number.
Execution Time: 0.0004298686981201172 seconds
PS C:\Users\rohit\OneDrive\Desktop\ai>

```

Task #9 – Few-Shot Prompting for Nested Dictionary Extraction

Objective

Use few-shot prompting (2–3 examples) to instruct the AI to create a function that parses a nested dictionary representing student information.

Requirements

- The function should extract and return:

- o Full Name
- o Branch
- o

SGPA

Expected Output

A reusable Python function that correctly navigates and extracts values from nested dictionaries based on the provided examples

```
# Parse student information from a nested dictionary and display full name, branch, and SGPA
```

```
student_info = {  
    'name': {  
        'first': 'John',  
        'last': 'Doe'  
    },  
    'branch': 'Computer Science',  
    'SGPA': 9.2  
}  
  
def parse_student_info(info):  
    full_name = f"{info['name']['first']} {info['name']['last']}"  
    branch = info['branch']  
    sgpa = info['SGPA']  
    return full_name, branch, sgpa  
  
full_name, branch, sgpa = parse_student_info(student_info)  
print(f"Full Name: {full_name}, Branch: {branch}, SGPA: {sgpa}")  
|
```

OUTPUT:

```
PS C:\Users\rohit\OneDrive\Desktop\ai> & C:/Users/rohit/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/rohit/OneDrive/Desktop/ai/new.py  
Full Name: John Doe, Branch: Computer Science, SGPA: 9.2  
PS C:\Users\rohit\OneDrive\Desktop\ai> |
```

Task Description #10 (Loops – Perfect Numbers in a Range)

Task: Generate a function using AI that displays all Perfect Numbers within a user-specified range (e.g., 1 to 1000).

Instructions:

- A Perfect Number is a positive integer equal to the sum of its proper divisors (excluding itself).
 - o Example: $6 = 1 + 2 + 3$, $28 = 1 + 2 + 4 + 7 + 14$.
- Use a for loop to find divisors of each number in the range.
- Validate correctness with known Perfect Numbers (6, 28, 496...).
- Ask AI to regenerate an optimized version (using divisor check only up to $\sqrt{n}$).

Expected Output #12:

- Python program that lists Perfect Numbers in the given range.
- Optimized version with explanation.

```
217 # Display all perfect numbers within the range 1-1000
218 # where a perfect number is equal to the sum of its proper divisors
219
220 def is_perfect_number(num):
221     if num < 2:
222         return False
223     divisors_sum = sum(i for i in range(1, num) if num % i == 0)
224     return divisors_sum == num
225
226 perfect_numbers = [num for num in range(1, 1001) if is_perfect_number(num)]
227 print("Perfect numbers between 1 and 1000 are:", perfect_numbers)
228
```

OUTPUT:

```
PS C:\Users\rohit\OneDrive\Desktop\ai> & C:/Users/rohit/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/rohit/OneDrive/Desktop/ai/new.py
● Perfect numbers between 1 and 1000 are: [6, 28, 496]
○ PS C:\Users\rohit\OneDrive\Desktop\ai>
```